

Name: WASEEM AKRAM
UID- 24BAI70006
Subject: ADBMS

Branch: BE-CSE-AIML
Semester: 5
Section- 24KRG-1 'A'

Experiment-08

Aim:

(A) Problem Statement

A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

The company wants to provide an order report to an external client. The report must contain the Order ID, Order Date, Customer Name, Product Name, and Final Amount.

For every order, first calculate the Gross Amount using:

$$\text{Gross Amount} = \text{Unit Price} \times \text{Quantity}$$

Then calculate the Discount Amount using:

$$\text{Discount Amount} = \text{Gross Amount} \times \text{Discount Percentage} / 100$$

Finally, calculate the Final Amount using:

$$\text{Final Amount} = \text{Gross Amount} - \text{Discount Amount}$$

Create a complex view using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.

The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user only SELECT permission on the view. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

(B) Problem Statement

An e-commerce company wants to regularly analyze the performance of its product categories.

The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple JOINS, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements.

Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.

Create a PostgreSQL Materialized View named: mv_category_sales

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Code and Output:

A.

```
Query Query History
69      P.product_name,
70
71      (P.unit_price * O.quantity)
72      -
73      (
74          (P.unit_price * O.quantity)
75          * O.discount_percent / 100
76      ) AS final_amount
77
78 FROM customer_master C
79
80 JOIN sales_orders O
81     ON C.customer_id = O.customer_id
82
83 JOIN product_catalog P
84     ON P.product_id = O.product_id;
85
86 EXPLAIN ANALYZE
87 SELECT * FROM vw_PRODUCT_DISCOUNT_INFO;
88
89
90 CREATE USER KRG_1A
91 PASSWORD '1234';
92
93 --GRANT AND REVOKE
94
95 GRANT SELECT ON vw_PRODUCT_DISCOUNT_INFO TO KRG_1A;

Data Output Messages Notifications

GRANT

Query returned successfully in 84 msec.
```

adbms_lab/krg_1a@PostgreSQL 18

Query Query History

```
1 SELECT * FROM vw_PRODUCT_DISCOUNT_INFO;
```

Data Output Messages Notifications

Showing rows: 1 to 7

	order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	finalAmount numeric
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.000000000000000000
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.000000000000000000
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.000000000000000000
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.000000000000000000
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.000000000000000000
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.000000000000000000
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.000000000000000000

B.

```
123
124
125 --display materialized view
126
127 EXPLAIN ANALYZE
128 SELECT *
129 FROM mv_category_sales_01
130 ORDER BY category_id;
131
```

Data Output Messages Notifications

Showing rows: 1 to 10

QUERY PLAN	
	text
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=1
5	-> Seq Scan on mv_category_sales_01 (cost=0.00..12.80 rows=280 width=262) (actual time=0.013..0.014 rows=3.00 loo...
6	Buffers: shared hit=1
7	Planning:
8	Buffers: shared hit=5
9	Planning Time: 1.116 ms
10	Execution Time: 0.057 ms

Total rows: 10 Query complete 00:00:00.096

```
65
66
67 --Solution
68 CREATE MATERIALIZED VIEW mv_category_sales_01
69 AS
70
71 SELECT
72     c.category_id,
73     c.category_name,
74     |
75     SUM(od.quantity) AS total_products_sold,
76
77     COUNT(DISTINCT od.order_id) AS total_orders,
78
79     COUNT(DISTINCT od.customer_id) AS unique_customers,
80
```

Data Output Messages Notifications

Showing rows: 1 to 6

QUERY PLAN	
	text
1	Seq Scan on mv_category_sales_01 (cost=0.00..12.80 rows=280 width=262) (actual time=0.010..0.011 rows=3.00 loo...
2	Buffers: shared hit=1
3	Planning:
4	Buffers: shared hit=21
5	Planning Time: 0.862 ms
6	Execution Time: 0.022 ms

